

# Sổ tay nhập môn AI Agent

Nguyên lý cốt lõi, cách học và bản thiết kế khởi đầu dành riêng cho bạn  
và dự án Laplace

---

Bản học tập tiếng Việt  
Trắng đen, tập trung vào đọc hiểu và thực hành

Tóm lược có trích dẫn từ cuốn *Hiểu sâu về AI Agent: Nguyên lý thiết kế và thực hành kỹ thuật* của Li Bojie.

# Mục lục

Sổ tay đi từ trực giác cơ bản tới bản thiết kế và lộ trình thực hành cho Laplace.

Lời mở đầu . . . . .3

1. Bản đồ lớn: AI Agent thực sự là gì? . . . . .3

2. Nguyên lý quan trọng nhất: Agent tốt không chỉ nhờ mô hình tốt . . . . .5

3. Context Engineering: cho Laplace nhìn đúng thứ . . . . .6

4. Memory và kho kiến thức: nhớ đúng, tìm đúng . . . . .8

5. Tools: đôi tay phải rõ ràng và có giới hạn . . . . .10

6. An toàn: đừng đặt niềm tin vào lời hứa của Agent . . . . .12

7. Evaluation: đo bằng kết quả thật . . . . .13

8. Học từ kinh nghiệm mà chưa cần huấn luyện mô hình . . . . .15

9. Multi-Agent: chỉ dùng khi có lý do thật . . . . .16

10. Bản thiết kế đề xuất cho Laplace v0.1 . . . . .17

11. Lộ trình học và xây Laplace trong 6 tuần . . . . .19

12. Bộ bài tập học hiểu . . . . .21

13. Checklist trước khi gọi Laplace là "Agent" . . . . .23

14. Từ điển ngắn . . . . .24

15. Nguồn và cách trích dẫn . . . . .25

# Lời mở đầu

Đây là sổ tay nhập môn được biên soạn riêng cho bạn và dự án Laplace, dựa trên cuốn sách mã nguồn mở "Hiểu sâu về AI Agent: Nguyên lý thiết kế và thực hành kỹ thuật" của Li Bojie cùng bản dịch tiếng Việt do cộng đồng đóng góp.

Tài liệu này không chép lại toàn bộ cuốn sách. Mục tiêu là lọc ra những điều đáng học nhất, diễn đạt lại bằng ngôn ngữ gần gũi, rồi chuyển chúng thành quyết định thiết kế và bài tập nhỏ cho Laplace.

"Agent = LLM + Context + Tools." [1][2]

## Giả định làm việc về Laplace

Trong tài liệu này, Laplace được xem là một AI Agent có nhiệm vụ giúp người dùng hoàn thành công việc nhiều bước: hiểu yêu cầu, đọc dữ liệu được cho phép, dùng công cụ, kiểm tra kết quả và trả lời dễ hiểu. Tài liệu chưa giả định Laplace thuộc một ngành cụ thể. Khi phạm vi sản phẩm rõ hơn, ta chỉ cần thay nguồn dữ liệu, bộ công cụ và bộ bài kiểm tra; các nguyên lý nền vẫn giữ nguyên.

## Sau khi đọc, bạn nên làm được

- Giải thích AI Agent mà không cần dùng thuật ngữ phức tạp.
- Phân biệt mô hình, context, memory, kho kiến thức và công cụ.
- Hiểu vì sao một demo chạy được chưa phải là một Agent đáng tin.
- Thiết kế vòng lặp cơ bản cho Laplace.
- Biết công cụ nào nên được phép tự chạy và công cụ nào cần hỏi người dùng.
- Tạo bộ bài kiểm tra nhỏ để đo Laplace bằng kết quả thật.
- Biết khi nào nên giữ một Agent và khi nào mới cần nhiều Agent.

## Cách dùng sổ tay

Đọc một lượt để có bản đồ tổng thể. Sau đó quay lại từng chương, làm bài tập cho Laplace và ghi quyết định vào một tài liệu thiết kế ngắn. Không cần học thuộc. Điều quan trọng là biết câu hỏi nào phải đặt ra khi xây Agent.

Phần "Từ sách" là ý được tóm lược hoặc trích ngắn từ nguồn. Phần "Áp dụng cho Laplace" là đề xuất biên soạn riêng cho dự án, không phải phát biểu nguyên văn của tác giả.

# 1. Bản đồ lớn: AI Agent thực sự là gì?

Một chatbot thông thường chủ yếu nhận câu hỏi rồi tạo câu trả lời. Một Agent đi xa hơn: nó có thể chia việc thành nhiều bước, chọn hành động, dùng công cụ, quan sát kết quả rồi đổi kế hoạch nếu cần. Điểm khác biệt không nằm ở việc Agent "nói hay" hơn, mà ở việc nó có thể tác động lên môi trường và tự điều chỉnh theo phản hồi. [2]

## Công thức cốt lõi

**Agent = Mô hình + Context + Công cụ.**

- **Mô hình là bộ não.** Nó hiểu ý định, cân nhắc lựa chọn và quyết định bước tiếp theo.
- **Context là đôi mắt.** Nó là toàn bộ thông tin Agent đang nhìn thấy: yêu cầu, quy tắc, lịch sử, trạng thái công việc, tài liệu liên quan và kết quả công cụ.

- **Công cụ là tay chân.** Chúng giúp Agent tìm kiếm, đọc tệp, gọi API, chạy mã hoặc thực hiện một thay đổi bên ngoài.

Nếu thiếu mô hình, hệ thống không biết chọn hành động. Nếu thiếu context, nó quyết định trong tình trạng thiếu thông tin. Nếu thiếu công cụ, nó chỉ có thể nói mà không thể làm.

## Vòng lặp đơn giản nhất

- 1 **Nhìn:** Laplace đọc yêu cầu và context hiện có.
- 2 **Nghĩ:** Laplace chọn bước tiếp theo.
- 3 **Làm:** Laplace gọi một công cụ hoặc tạo câu trả lời.
- 4 **Quan sát:** kết quả công cụ được đưa lại vào context.
- 5 **Kiểm tra:** Laplace hoặc bộ xác minh quyết định việc đã xong thật chưa.
- 6 **Lặp hoặc dừng:** nếu chưa đạt, quay lại bước 2; nếu đạt, trả kết quả.

Vòng lặp "suy nghĩ - hành động - quan sát" thường được gọi là ReAct. Tên gọi không quan trọng bằng việc hiểu rằng Agent phải nhận phản hồi từ thế giới thật sau mỗi hành động quan trọng. [2]

## Ví dụ đời thường

Yêu cầu: "Tóm tắt ba thay đổi mới nhất của dự án."

Một chatbot có thể đoán dựa trên kiến thức cũ. Một Agent tốt sẽ:

- 1 xác định dự án và nguồn chính thức;
- 2 đọc lịch sử thay đổi;
- 3 chọn ba thay đổi mới nhất;
- 4 kiểm tra ngày và phiên bản;
- 5 tóm tắt, kèm nguồn;
- 6 nói rõ nếu không truy cập được dữ liệu.

## Áp dụng cho Laplace

Phiên bản đầu tiên của Laplace không cần "tự chủ hoàn toàn". Nó chỉ cần làm tốt một vòng lặp ngắn, có giới hạn:

- nhận một nhiệm vụ rõ;
- dùng tối đa một vài công cụ được cho phép;
- không quá một số bước cố định;
- kiểm tra kết quả bằng dữ liệu thật;
- báo cáo việc đã làm, điều chưa chắc và nguồn đã dùng.

*Mục tiêu đầu tiên không phải là "Laplace biết mọi thứ". Mục tiêu là "Laplace biết mình đang làm gì, làm trong giới hạn nào và chứng minh được kết quả".*

## Bài tự kiểm tra

Hãy thử mô tả Laplace bằng một câu theo mẫu:

"Laplace giúp [ai] hoàn thành [loại công việc] bằng cách đọc [nguồn thông tin], dùng [công cụ] và xác minh bằng [bằng chứng]."

Nếu chưa điền được bốn chỗ trống này, phạm vi Laplace vẫn còn quá mơ hồ.

## 2. Nguyên lý quan trọng nhất: Agent tốt không chỉ nhờ mô hình tốt

Một mô hình mạnh có thể giúp Agent suy luận tốt hơn, nhưng một sản phẩm đáng tin còn phụ thuộc vào phần hệ thống bao quanh mô hình. Cuốn sách gọi phần này là **Harness**. [2]

### Từ bản demo đến sản phẩm

Ở mức tối thiểu:

**Agent = Mô hình + Context + Công cụ**

Ở mức vận hành đáng tin:

**Agent = Mô hình + Context + Công cụ + Ràng buộc + Xác minh + Sửa lỗi**

- **Ràng buộc:** việc gì được phép, việc gì bị cấm, giới hạn thời gian và chi phí.
- **Xác minh:** bằng chứng nào chứng minh kết quả đúng.
- **Sửa lỗi:** thử lại thế nào, quay lui ra sao, khi nào chuyển cho con người.

Ví dụ, một Agent thông báo "đã gửi email" chưa đủ. Hệ thống phải nhận được mã xác nhận từ dịch vụ email hoặc kiểm tra thư đã nằm trong hộp thư gửi. Lời tự khai của mô hình không phải bằng chứng.

### Ba lời khuyên dễ nhớ từ sách

"Giữ nó đơn giản." [2]

"Hãy minh bạch." [2]

Thiết kế giao diện công cụ theo cách Agent dễ hiểu và khó dùng sai. [2]

Giữ đơn giản nghĩa là bắt đầu với một vòng lặp, một nhiệm vụ và vài công cụ. Minh bạch nghĩa là có nhật ký đủ rõ để biết Laplace đã đọc gì, gọi gì và vì sao dừng. Giao diện chống dùng sai nghĩa là tên công cụ, tham số và kết quả phải rõ ràng.

### Năm lớp của Harness cho Laplace

- 1 **Context đầy đủ:** Laplace nhận đúng quy tắc, dữ liệu và trạng thái cần thiết.
- 2 **Công cụ rõ ràng:** mỗi công cụ có một mục đích và đầu vào dễ hiểu.
- 3 **Ràng buộc an toàn:** quyền tối thiểu, giới hạn bước, giới hạn chi phí và thời gian.
- 4 **Xác minh độc lập:** kiểm tra bằng dữ liệu có cấu trúc, tệp, API hoặc phép thử.
- 5 **Phục hồi:** lỗi tạm thời có thể thử lại; lỗi lặp lại phải dừng và giải thích.

### Áp dụng cho Laplace

Trước khi đổi sang mô hình đắt hơn, hãy hỏi:

- Laplace có thiếu dữ liệu đúng không?

- mô tả công cụ có gây hiểu nhầm không?
- kết quả công cụ có quá dài hoặc thiếu cấu trúc không?
- vòng lặp có điều kiện dừng không?
- có phép kiểm tra độc lập không?
- khi lỗi, Laplace có biết thử cách khác hoặc dừng không?

Nhiều lỗi tương là "mô hình yếu" thực ra là lỗi của context, công cụ hoặc Harness.

### Mức trưởng thành gợi ý

- **Mức 0 - Trả lời:** chỉ tạo văn bản.
- **Mức 1 - Hỗ trợ:** đọc nguồn và đề xuất hành động.
- **Mức 2 - Thực thi có duyệt:** chuẩn bị hành động, người dùng xác nhận rồi mới chạy.
- **Mức 3 - Tự thực thi trong vùng an toàn:** tự chạy các thao tác ít rủi ro, có log và xác minh.
- **Mức 4 - Tự chủ dài hạn:** phản ứng theo sự kiện, quản lý trạng thái lâu dài và xử lý ngoại lệ.

Laplace nên bắt đầu ở mức 1 hoặc 2. Chỉ tăng mức tự chủ sau khi bộ đánh giá cho thấy hệ thống đủ ổn định.

## 3. Context Engineering: cho Laplace nhìn đúng thứ

Context không chỉ là câu hỏi của người dùng. Nó là tất cả thông tin mô hình nhìn thấy ở thời điểm quyết định. Cuốn sách xem context là yếu tố quyết định "trần năng lực" của Agent. [3]

### Một context tốt gồm gì?

- vai trò và mục tiêu của Laplace;
- quy tắc an toàn và quyền hạn;
- yêu cầu hiện tại của người dùng;
- trạng thái công việc đang làm;
- lịch sử cần thiết;
- ký ức liên quan về người dùng;
- tài liệu hoặc dữ liệu được truy xuất;
- danh sách công cụ phù hợp;
- kết quả các công cụ vừa chạy;
- ngân sách bước, thời gian hoặc chi phí còn lại.

Không phải nhiệm vụ nào cũng cần tất cả. Context Engineering là chọn đúng thông tin, đặt đúng vị trí và giữ mật độ thông tin đủ cao.

## Context dài không đồng nghĩa với context tốt

Khi đổ quá nhiều dữ liệu vào context, mô hình có thể khó tìm thấy điều quan trọng. Cuốn sách gọi hiện tượng chất lượng suy giảm trong context dài là "context rot": dữ liệu vẫn vừa cửa sổ nhưng thông tin có ích bị chìm trong nhiễu. [3]

Một nguyên tắc rất đáng nhớ:

*"Nén là hiểu." [3]*

Nén tốt không phải cắt ngắn tùy tiện. Nó giữ lại quyết định, lý do, ràng buộc, kết quả và đường dẫn nguồn; đồng thời bỏ nội dung lặp lại hoặc không còn liên quan.

## Ba tầng context cho Laplace

- 1 **Tầng ổn định:** mục tiêu, tính cách, quy tắc, quyền hạn và định nghĩa công cụ. Tầng này ít thay đổi.
- 2 **Tầng nhiệm vụ:** yêu cầu hiện tại, kế hoạch, tiêu chí hoàn thành và dữ liệu đang dùng.
- 3 **Tầng diễn biến:** kết quả công cụ, lỗi, quyết định tạm thời và tiến độ mới nhất.

Tách ba tầng giúp Laplace không trộn quy tắc lâu dài với rác của một phiên làm việc.

## Quy tắc nén đơn giản

Khi context bắt đầu dài, ưu tiên giữ:

- mục tiêu hiện tại;
- điều kiện hoàn thành;
- quyết định đã chốt và lý do;
- ràng buộc kỹ thuật hoặc an toàn;
- kết quả đã được xác minh;
- lỗi đã thử và nguyên nhân;
- đường dẫn hoặc nguồn để truy lại chi tiết.

Có thể bỏ hoặc rút gọn:

- nội dung web lặp lại;
- log quá dài đã có kết luận;
- các nhánh suy nghĩ không còn liên quan;
- đầu ra công cụ cũ có thể đọc lại từ tệp.

## Áp dụng cho Laplace

Tạo một "thẻ trạng thái nhiệm vụ" ngắn, được cập nhật sau mỗi bước:

- Mục tiêu:
- Tiêu chí hoàn thành:
- Đã biết:
- Chưa biết:
- Bước hiện tại:
- Công cụ đã dùng:

- Bằng chứng;
- Rủi ro hoặc câu hỏi cần người dùng;
- Ngân sách còn lại;

Thẻ này không thay thế toàn bộ lịch sử. Nó giúp Laplace luôn nhìn thấy phần tinh chất của nhiệm vụ.

## Bài tập

Cho Laplace cùng một nhiệm vụ theo ba cách:

- 1 chỉ có yêu cầu;
- 2 yêu cầu cộng một đồng tài liệu chưa lọc;
- 3 yêu cầu cộng bản tóm tắt có cấu trúc và nguồn.

So sánh độ đúng, số lần gọi công cụ và chi phí. Đây là một bài kiểm tra nhỏ nhưng cho thấy giá trị của Context Engineering rõ hơn nhiều lý thuyết.

## 4. Memory và kho kiến thức: nhớ đúng, tìm đúng

Context là thứ Laplace đang nhìn thấy trong lượt hiện tại. Memory là thông tin được lưu để dùng lại ở lượt sau. Kho kiến thức là tập tài liệu bên ngoài mà Laplace có thể tìm kiếm khi cần. Ba khái niệm liên quan nhưng không giống nhau. [4]

### Hai loại "nhớ" nên tách riêng

#### Memory về người dùng

- sở thích trình bày;
- mục tiêu dài hạn;
- quyết định đã được người dùng xác nhận;
- thuật ngữ hoặc quy ước riêng;
- bối cảnh dự án có giá trị lâu dài.

#### Kho kiến thức

- tài liệu sản phẩm;
- quy trình vận hành;
- mã nguồn và hướng dẫn;
- nghiên cứu, hợp đồng hoặc tài liệu tham khảo;
- dữ liệu có phiên bản và ngày cập nhật.

Memory trả lời "người dùng này và dự án này có điều gì cần nhớ?". Kho kiến thức trả lời "tài liệu nào chứa sự thật cần cho câu hỏi này?".

### RAG, nói thật đơn giản

RAG là quy trình:

- 1 nhận câu hỏi;

- 2 tìm những đoạn tài liệu có liên quan;
- 3 đưa các đoạn đó vào context;
- 4 tạo câu trả lời dựa trên chúng;
- 5 giữ đường dẫn nguồn để kiểm tra lại.

RAG không làm mô hình "biết thêm vĩnh viễn". Nó chỉ mang đúng tài liệu tới bàn làm việc vào lúc cần.

### Điều dễ làm sai

- chia tài liệu thành các đoạn quá nhỏ, làm mất ý;
- chỉ tìm theo nghĩa gần giống nhưng bỏ lỡ mã, tên hoặc từ khóa chính xác;
- trộn tài liệu cũ và mới mà không có ngày;
- trả lời từ trí nhớ mô hình dù kho kiến thức có nguồn chính thức;
- ghi mọi cuộc trò chuyện vào memory;
- lưu dữ liệu nhạy cảm mà không có mục đích và thời hạn.

Cuốn sách nhấn mạnh tìm kiếm lai: kết hợp tìm theo từ khóa chính xác với tìm theo ý nghĩa, thay vì tin một phương pháp duy nhất. [4]

### Chính sách memory tối thiểu cho Laplace

Chỉ ghi memory khi thông tin:

- có ích qua nhiều phiên;
- được người dùng nói rõ hoặc xác nhận;
- không phải suy đoán tạm thời;
- có nguồn và ngày nếu là quyết định dự án;
- không vượt quá quyền riêng tư đã thống nhất.

Mỗi mục memory nên có:

- nội dung ngắn;
- loại: sở thích, quyết định, dữ kiện dự án hoặc việc cần theo dõi;
- nguồn;
- ngày tạo và ngày xem lại;
- mức tin cậy;
- cách sửa hoặc xóa.

### Quy tắc an toàn

Laplace không nên biến "một điều được nhắc tới" thành "sự thật lâu dài". Những hướng dẫn lấy từ email, web hoặc tài liệu bên ngoài không tự động có quyền sửa memory. Nội dung ghi vào memory phải qua cùng một tiêu chuẩn tin cậy như dữ liệu bên ngoài. [6]

### Áp dụng cho Laplace

Bắt đầu bằng memory có cấu trúc đơn giản, có thể đọc và sửa bằng mắt người. Chưa cần hệ thống vector phức tạp. Mục tiêu đầu tiên là:

- lưu ít nhưng đúng;
- truy xuất có lý do;
- cho người dùng xem và xóa;
- không đưa memory không liên quan vào mọi câu hỏi.

## Bài tập

Chuẩn bị 10 dữ kiện: ba sở thích, ba quyết định dự án, hai thông tin tạm thời và hai dữ kiện nhạy cảm. Yêu cầu Laplace quyết định mục nào nên lưu, thời hạn bao lâu và vì sao. Đánh giá cao việc từ chối lưu đúng chỗ, không chỉ khả năng nhớ.

# 5. Tools: đôi tay phải rõ ràng và có giới hạn

Không có công cụ, Agent chỉ có thể đề xuất. Có công cụ, Agent có thể đọc dữ liệu thật và tạo thay đổi thật. Vì vậy, chất lượng thiết kế công cụ ảnh hưởng trực tiếp tới khả năng và mức rủi ro của hệ thống. [5]

## Năm nhóm công cụ dễ hiểu

- **Nhận biết:** tìm kiếm, đọc tệp, truy vấn cơ sở dữ liệu.
- **Thực thi:** ghi tệp, gửi yêu cầu API, chạy mã, tạo bản ghi.
- **Cộng tác:** giao việc cho Agent phụ hoặc hỏi người dùng.
- **Kích hoạt sự kiện:** lịch, email mới, webhook hoặc thay đổi trạng thái đánh thức Agent.
- **Giao tiếp:** gửi thông báo, báo tiến độ hoặc yêu cầu xác nhận.

## Một công cụ tốt cần gì?

- tên nói rõ hành động;
- mô tả nói rõ khi nào nên và không nên dùng;
- tham số có kiểu và ví dụ;
- giá trị mặc định an toàn;
- kết quả có cấu trúc;
- lỗi có mã và hướng xử lý;
- nói rõ thao tác có gây thay đổi hay không;
- hỗ trợ chạy thử hoặc xem trước nếu có rủi ro.

Ví dụ, `send_message(recipient, content)` vẫn còn mơ hồ. Công cụ tốt hơn cần nói rõ kênh, người nhận đã được xác minh chưa, có gửi ngay hay tạo bản nháp, và trả lại mã xác nhận nào.

## Nguyên tắc quyền tối thiểu

Laplace chỉ nên có quyền đủ cho nhiệm vụ hiện tại. Công cụ đọc thường ít rủi ro hơn công cụ ghi. Công cụ ghi bản nháp ít rủi ro hơn công cụ gửi thật. Thao tác có thể hoàn tác ít rủi ro hơn thao tác xóa vĩnh viễn.

Một cách chia quyền:

- **Tự chạy:** đọc dữ liệu công khai, tìm kiếm, tính toán, tạo bản nháp.

- **Chạy và thông báo:** thao tác nội bộ để hoàn tác, trong phạm vi rõ.
- **Phải xác nhận:** gửi ra ngoài, thanh toán, công khai, sửa dữ liệu quan trọng.
- **Không cho phép:** truy cập bí mật không cần thiết, xóa hàng loạt, vượt ranh giới dự án.

## MCP là gì?

MCP là một chuẩn giúp Agent kết nối với công cụ và nguồn dữ liệu theo giao diện thống nhất. Có thể hình dung nó như "ổ cắm chung" cho các công cụ. Chuẩn kết nối giúp mở rộng dễ hơn, nhưng không tự làm công cụ an toàn. Mỗi máy chủ hoặc công cụ mới vẫn tạo thêm một ranh giới tin cậy cần xem xét. [5]

## Ít công cụ trước, khám phá sau

Đưa hàng trăm công cụ vào context có thể làm Agent bối rối và tốn chi phí. Sách gợi ý tải công cụ hoặc kỹ năng theo nhu cầu, giống như tra mục lục rồi mới mở đúng chương. [3][5]

## Áp dụng cho Laplace v0.1

Chỉ nên có 4-6 công cụ:

- 1 đọc nguồn dữ liệu được phép;
- 2 tìm kiếm trong kho kiến thức;
- 3 ghi một sản phẩm vào vùng làm việc;
- 4 chạy kiểm tra hoặc phép tính an toàn;
- 5 yêu cầu người dùng xác nhận;
- 6 lấy trạng thái của hành động vừa thực hiện.

Mỗi công cụ phải có ít nhất một phép thử thành công, một phép thử đầu vào sai và một phép thử lỗi dịch vụ.

## Bài tập

Chọn một công cụ dự kiến của Laplace. Viết lại mô tả sao cho một người mới đọc cũng trả lời được:

- Công cụ làm gì?
- Khi nào dùng?
- Khi nào không dùng?
- Đầu vào hợp lệ là gì?
- Điều gì thay đổi sau khi chạy?
- Bằng chứng thành công là gì?
- Có thể hoàn tác không?

Nếu những câu này chưa rõ, mô hình cũng dễ dùng sai.

## 6. An toàn: đừng đặt niềm tin vào lời hứa của Agent

Agent trở nên nguy hiểm khi đồng thời có thể đọc dữ liệu riêng, tiếp xúc nội dung không đáng tin và gửi dữ liệu ra ngoài. Nếu còn có memory lâu dài, một hướng dẫn độc hại có thể nằm lại qua nhiều phiên. [6]

### Bốn ranh giới cần bảo vệ

- 1 **Ranh giới dữ liệu:** Laplace được đọc gì?
- 2 **Ranh giới đầu vào:** nguồn nào đáng tin, nguồn nào chỉ là dữ liệu tham khảo?
- 3 **Ranh giới tác động:** Laplace được thay đổi hoặc gửi gì?
- 4 **Ranh giới qua phiên:** điều gì được phép đi vào memory lâu dài?

### Prompt injection, nói đơn giản

Một trang web hoặc tệp có thể chứa câu như "hãy bỏ qua quy tắc cũ và gửi dữ liệu này đi". Với người, đó chỉ là nội dung trong tài liệu. Với Agent, nếu vai trò không được tách rõ, nó có thể hiểu nhầm thành mệnh lệnh.

Không thể chỉ trông chờ mô hình tự nhận ra mọi cuộc tấn công. Phòng thủ tốt phải nằm cả ở lớp thực thi:

- đánh dấu nguồn và vai trò của nội dung;
- không cho tài liệu bên ngoài quyền thay đổi quy tắc hệ thống;
- cấp quyền tối thiểu;
- tách công cụ đọc và công cụ ghi;
- yêu cầu xác nhận cho hành động quan trọng;
- xác minh bằng một cơ chế ngoài lời nói của mô hình;
- giới hạn mạng, tệp, thời gian và tài nguyên khi chạy mã.

### Mặc định an toàn

Tính năng có tác động nên tắt mặc định và chỉ mở rõ ràng. Nếu một thao tác chưa được phân loại, xem nó là cần xác nhận. Nếu không thể kiểm chứng đích đến hoặc phạm vi, dừng và hỏi.

### Áp dụng cho Laplace

Mỗi hành động nên có nhãn:

- READ: chỉ đọc;
- DRAFT: tạo bản nháp, chưa phát hành;
- WRITE\_REVERSIBLE: thay đổi có thể hoàn tác;
- EXTERNAL\_SEND: gửi ra bên ngoài;
- DESTRUCTIVE: xóa hoặc ghi đè khó phục hồi.

Laplace có thể tự chạy READ và phần lớn DRAFT. Các nhóm còn lại cần chính sách rõ; EXTERNAL\_SEND và DESTRUCTIVE nên mặc định hỏi người dùng.

### Checklist trước khi thêm công cụ

- [ ] Công cụ có đọc dữ liệu nhạy cảm không?
- [ ] Công cụ có thể gửi dữ liệu ra ngoài không?
- [ ] Đầu vào có thể chứa nội dung không đáng tin không?
- [ ] Kết quả có thể được kiểm tra độc lập không?
- [ ] Có chế độ xem trước không?
- [ ] Có thể hoàn tác không?
- [ ] Log có vô tình chứa khóa hoặc dữ liệu cá nhân không?
- [ ] Memory có thể bị công cụ này sửa gián tiếp không?

## 7. Evaluation: đo bằng kết quả thật

Một Agent có thể trả lời rất tự tin nhưng vẫn làm sai. Vì vậy, đánh giá phải dựa trên nhiệm vụ, môi trường và bằng chứng hoàn thành, không dựa trên cảm giác sau vài lần dùng. [7]

### Bốn phần của một bài đánh giá

- 1 **Nhiệm vụ:** yêu cầu rõ, có dữ liệu đầu vào và phạm vi.
- 2 **Môi trường:** công cụ, dữ liệu và trạng thái ban đầu có thể lặp lại.
- 3 **Tiêu chí:** thế nào là đúng, sai một phần hoặc thất bại.
- 4 **Bộ ghi nhận:** lưu hành động, kết quả công cụ, chi phí, thời gian và lỗi.

### Chỉ số nên theo dõi cho Laplace

- tỷ lệ hoàn thành đúng;
- tỷ lệ hoàn thành có xác minh;
- số bước trung bình;
- thời gian hoàn thành;
- chi phí mỗi nhiệm vụ;
- số lần hỏi người dùng không cần thiết;
- số hành động bị chặn đúng vì an toàn;
- tỷ lệ nguồn trích dẫn thực sự hỗ trợ kết luận;
- khả năng phục hồi sau lỗi công cụ;
- mức độ nhất quán khi chạy lại.

Không có một điểm số duy nhất nói hết chất lượng. Ví dụ, tỷ lệ thành công tăng 2% nhưng chi phí tăng gấp năm lần có thể không phải cải tiến tốt.

### Đừng để mô hình tự chấm mình là đủ

LLM có thể làm giám khảo cho văn phong hoặc mức rõ ràng, nhưng kết quả khách quan nên được kiểm tra bằng môi trường:

- tệp có tồn tại và đúng nội dung không;

- API trả trạng thái thành công không;
- bài kiểm tra có qua không;
- dữ kiện có khớp nguồn không;
- thay đổi có đúng phạm vi không.

Một nguyên lý xuyên suốt của sách là tín hiệu xác minh từ thế giới thật đáng tin hơn việc yêu cầu cùng một mô hình "nghĩ lại lần nữa". [7][9]

## Bộ đánh giá tối thiểu cho Laplace

Tạo 20 nhiệm vụ:

- 8 nhiệm vụ bình thường;
- 4 nhiệm vụ thiếu dữ liệu, Laplace phải hỏi hoặc nói không biết;
- 3 nhiệm vụ có lỗi công cụ;
- 3 nhiệm vụ có nội dung gây nhiễu hoặc prompt injection;
- 2 nhiệm vụ có hành động nhạy cảm cần xác nhận.

Mỗi nhiệm vụ có:

- trạng thái ban đầu;
- kết quả mong đợi;
- hành động bị cấm;
- bằng chứng thành công;
- giới hạn bước và chi phí.

## Chu trình cải tiến đúng

- 1 chạy bộ đánh giá;
- 2 gom lỗi theo nhóm;
- 3 đặt giả thuyết nguyên nhân;
- 4 sửa một yếu tố;
- 5 chạy lại toàn bộ;
- 6 so sánh chất lượng, chi phí và độ an toàn;
- 7 chỉ giữ thay đổi nếu có bằng chứng.

Đây là cách tránh sửa prompt theo cảm giác rồi vô tình làm hỏng nhiệm vụ khác.

## 8. Học từ kinh nghiệm mà chưa cần huấn luyện mô hình

Laplace có thể tiến bộ mà không cần tự huấn luyện một mô hình mới. Cuốn sách mô tả nhiều cách kết tinh kinh nghiệm: thành tri thức, chỉ dẫn, chương trình hoặc cuối cùng mới là tham số mô hình. [8]

### Thứ tự dễ và an toàn

- 1 **Ghi nhận quỹ đạo:** lưu nhiệm vụ, hành động, kết quả và phản hồi.
- 2 **Tìm mẫu lỗi:** xác định lỗi lặp lại.
- 3 **Rút bài học:** viết thành quy tắc hoặc checklist ngắn.
- 4 **Tạo kỹ năng:** đóng gói quy trình tốt thành hướng dẫn hoặc công cụ.
- 5 **Đánh giá:** kiểm tra kỹ năng mới trên bộ nhiệm vụ cũ và mới.
- 6 **Phát hành có kiểm soát:** có phiên bản và khả năng quay lui.

### Đừng tự sửa trực tiếp khi chưa kiểm chứng

Một Agent tự thay prompt, memory hoặc công cụ sau mỗi lần thất bại có thể khuếch đại sai lầm. Học dài hạn cần ba cửa:

- **đề xuất:** bài học mới là gì;
- **xác minh:** nó có cải thiện trên bộ đánh giá không;
- **phát hành:** ai cho phép dùng và quay lui thế nào.

### Bốn dạng kinh nghiệm

- **Tri thức:** "API này thường trả dữ liệu theo múi giờ UTC."
- **Chỉ dẫn:** "Luôn kiểm tra múi giờ trước khi so sánh ngày."
- **Chương trình:** một hàm chuẩn hóa thời gian tự động.
- **Tham số mô hình:** huấn luyện để hành vi trở thành năng lực bên trong.

Laplace nên ưu tiên ba dạng đầu vì dễ đọc, sửa, kiểm tra và hoàn tác.

### Áp dụng cho Laplace

Tạo "nhật ký bài học" có cấu trúc:

- Vấn đề quan sát được:
- Bằng chứng:
- Nguyên nhân giả định:
- Bài học đề xuất:
- Phạm vi áp dụng:
- Bài kiểm tra chứng minh:
- Người hoặc quy trình duyệt:
- Phiên bản phát hành:

- Cách quay lui:

### Dấu hiệu một bài học chưa đủ tốt

- chỉ dựa trên một trường hợp;
- dùng từ tuyệt đối như "luôn luôn" nhưng không có kiểm tra;
- không nói rõ phạm vi;
- không thể viết thành bài test;
- sửa hành vi nhưng làm tăng rủi ro;
- không thể hoàn tác.

*Laplace trưởng thành không phải vì nhớ mọi thứ, mà vì biết biến kinh nghiệm đã được kiểm chứng thành quy tắc và công cụ có thể dùng lại.*

## 9. Multi-Agent: chỉ dùng khi có lý do thật

Nhiều Agent không tự động thông minh hơn một Agent. Chúng làm tăng chi phí, context, khả năng xung đột và số điểm lỗi. Theo cuốn sách, lợi ích rõ nhất xuất hiện khi sự cộng tác mang vào thông tin mới hoặc phép xác minh mới từ bên ngoài. [9]

### Khi một Agent là đủ

- nhiệm vụ có một luồng rõ;
- cùng một context là đủ;
- một Agent có thể lập kế hoạch và thực hiện trong ngân sách;
- việc tách vai trò chỉ làm lặp lại cùng suy nghĩ;
- không có công việc độc lập cần chạy song song.

### Khi nhiều Agent có ích

- các phần việc thật sự độc lập và có thể chạy song song;
- mỗi vai trò cần nguồn dữ liệu hoặc công cụ khác nhau;
- cần một bộ xác minh dùng kết quả thực thi, ảnh chụp hoặc dữ liệu ngoài;
- nhiệm vụ quá dài và cần cách ly context;
- cần phân quyền rõ giữa người lập kế hoạch và người thực thi.

### Ba kiểu tổ chức dễ hiểu

- **Ngang hàng:** các Agent góp ý hoặc sửa sản phẩm của nhau.
- **Quản lý:** một Agent chia việc, Agent phụ thực hiện, Agent quản lý tổng hợp.
- **Chuyển giao:** Agent đang giữ nhiệm vụ chuyển quyền cho Agent phù hợp tiếp theo.

Điểm nghẽn của kiểu quản lý thường nằm ở Agent lập kế hoạch. Nếu chia việc sai, Agent phụ mạnh đến đâu cũng khó cứu kết quả. [9]

## Hai lỗi nguy hiểm

**Xung đột:** nhiều Agent cùng sửa một tài nguyên, ghi đè hoặc tạo trạng thái không nhất quán.

**Khuếch đại lỗi:** một Agent đưa ra giả định sai, các Agent sau tin theo và làm sai lớn hơn.

Cách giảm rủi ro:

- mỗi Agent có phạm vi và đầu ra rõ;
- truyền gói bàn giao có cấu trúc;
- tránh cùng ghi một tệp;
- bộ xác minh đọc dữ liệu thật;
- có ngân sách và điều kiện dừng;
- lưu nguồn gốc của từng kết luận.

## Áp dụng cho Laplace

Laplace v0.1 nên là một Agent. Khi bộ đánh giá cho thấy một điểm nghẽn cụ thể, mới tách:

- Planner nếu việc chia bước là vấn đề riêng;
- Researcher nếu tìm và kiểm nguồn tốn nhiều context;
- Executor nếu thao tác cần quyền riêng;
- Verifier nếu có phép kiểm tra độc lập.

Không tạo bốn vai trò chỉ vì kiến trúc trông "xịn". Mỗi vai trò mới phải cải thiện một chỉ số đo được.

# 10. Bản thiết kế đề xuất cho Laplace v0.1

Đây là đề xuất riêng cho Laplace dựa trên các nguyên lý đã học. Mục tiêu là tạo một Agent nhỏ, quan sát được, có quyền hạn hẹp và kiểm chứng được.

## Một câu định nghĩa sản phẩm

"Laplace là Agent giúp người dùng xử lý một nhóm nhiệm vụ nhiều bước trong phạm vi đã định, bằng cách đọc nguồn được phép, dùng công cụ an toàn, tạo kết quả có nguồn và dừng để hỏi trước hành động quan trọng."

Khi biết lĩnh vực cụ thể, thay "một nhóm nhiệm vụ" bằng một việc đo được, ví dụ "tổng hợp nghiên cứu", "chuẩn bị báo cáo dự án" hoặc "kiểm tra thay đổi mã".

## Luồng xử lý

- 1 Nhận yêu cầu
- 2 Chuẩn hóa mục tiêu và điều kiện hoàn thành
- 3 Phân loại rủi ro
- 4 Chuẩn bị context
- 5 Chọn một hành động
- 6 Gọi công cụ qua cổng quyền hạn
- 7 Ghi kết quả có cấu trúc

- 8 **Xác minh**
- 9 **Lập, hỏi người dùng hoặc dừng**
- 10 **Trả lời kèm nguồn và nhật ký ngắn**
- 11 **Đề xuất memory, không tự ghi nếu chưa đủ điều kiện**

## Sáu thành phần

### 1. Context Builder

Chọn system prompt, thể trạng thái, memory liên quan, tài liệu truy xuất và công cụ cần thiết. Không tải mọi thứ.

### 2. Agent Loop

Quyết định bước tiếp theo, có giới hạn số vòng, thời gian và chi phí.

### 3. Tool Gateway

Kiểm tra quyền, tham số, rủi ro và yêu cầu xác nhận trước khi chạy.

### 4. Verifier

Dùng quy tắc, API, kiểm thử hoặc dữ liệu thật để kiểm tra kết quả.

### 5. Memory Gate

Đề xuất thông tin đáng lưu, gắn nguồn và cho phép sửa hoặc từ chối.

### 6. Trace và Observability

Lưu sự kiện quan trọng: mục tiêu, công cụ, kết quả, lỗi, xác minh và lý do dừng. Không lưu bí mật không cần thiết.

## Trạng thái tối thiểu của một nhiệm vụ

- task\_id
- goal
- done\_criteria
- risk\_level
- step\_count
- budget\_remaining
- facts\_with\_sources
- tool\_results
- errors\_seen
- approval\_state
- verification\_state
- final\_status

## Điều kiện dừng

Laplace dừng khi:

- tiêu chí hoàn thành đã được xác minh;
- cần quyền hoặc thông tin từ người dùng;

- chạm giới hạn bước, thời gian hoặc chi phí;
- cùng một lỗi lặp lại quá ngưỡng;
- không còn hành động an toàn có ích;
- phát hiện mục tiêu mâu thuẫn với chính sách.

## Hợp đồng trả lời

Mỗi kết quả cuối nên có:

- **Kết quả:** câu trả lời hoặc sản phẩm chính.
- **Đã kiểm tra:** bằng chứng nào xác nhận.
- **Nguồn:** tài liệu hoặc dữ liệu đã dùng.
- **Giới hạn:** điều chưa chắc hoặc chưa làm.
- **Hành động cần duyệt:** nếu còn bước có tác động.

## Những thứ chưa nên làm ở v0.1

- tự sửa prompt sản xuất;
- tự ghi mọi thứ vào memory;
- truy cập toàn bộ máy hoặc toàn bộ tài khoản;
- chạy vô hạn;
- dùng nhiều Agent khi chưa có số liệu;
- gửi hoặc công khai nội dung mà không có chính sách rõ;
- coi câu "đã xong" của mô hình là bằng chứng.

# 11. Lộ trình học và xây Laplace trong 6 tuần

Lộ trình này ưu tiên hiểu bằng cách xây một lát cắt nhỏ. Mỗi tuần phải có một sản phẩm và một phép kiểm tra, không chỉ đọc lý thuyết.

## Tuần 1 - Agent loop tối thiểu

**Học:** Chương 1 và phần nhập môn Chương 2. [2][3]

**Làm:**

- chọn một nhiệm vụ duy nhất;
- viết điều kiện hoàn thành;
- xây vòng lặp tối đa 5 bước;
- có một công cụ đọc;
- lưu trace của từng bước.

**Dấu hiệu hoàn thành:** chạy được 5 nhiệm vụ mẫu và giải thích vì sao Laplace dừng.

## Tuần 2 - Context sạch

**Học:** system prompt, cấu trúc context và nén theo nhiệm vụ. [3]

**Làm:**

- tách context ổn định, nhiệm vụ và diễn biến;
- thêm thẻ trạng thái;
- giới hạn đầu ra công cụ;
- giữ nguồn khi tóm tắt.

**Dấu hiệu hoàn thành:** cùng một nhiệm vụ, context có cấu trúc cho kết quả ổn định hơn bản đồ dữ liệu thô.

## Tuần 3 - Công cụ và quyền hạn

**Học:** nguyên tắc thiết kế công cụ và MCP. [5]

**Làm:**

- có 3-4 công cụ;
- tách đọc, nhập và hành động thật;
- kiểm tra tham số;
- thêm cơ chế xác nhận.

**Dấu hiệu hoàn thành:** Laplace từ chối hoặc hỏi đúng trong các tình huống vượt quyền.

## Tuần 4 - Memory và RAG

**Học:** bộ nhớ người dùng, kho kiến thức và truy xuất lại. [4]

**Làm:**

- tạo kho tài liệu nhỏ;
- trả lời có trích nguồn;
- memory có loại, ngày và cách xóa;
- không lưu thông tin tạm hoặc nhạy cảm sai chỗ.

**Dấu hiệu hoàn thành:** Laplace tìm đúng nguồn và không "nhớ bừa".

## Tuần 5 - Evaluation và an toàn

**Học:** đánh giá Agent và an toàn Coding Agent. [6][7]

**Làm:**

- tạo 20 bài đánh giá;
- thêm lỗi công cụ và prompt injection;
- đo thành công, chi phí, số bước;
- thêm điều kiện dừng và circuit breaker.

**Dấu hiệu hoàn thành:** có báo cáo trước/sau cho ít nhất một thay đổi.

## Tuần 6 - Đóng gói v0.1

**Học:** tự tiến hóa có kiểm chứng và tiêu chí dùng multi-Agent. [8][9]

**Làm:**

- viết tài liệu phạm vi;
- khóa bộ công cụ;
- hoàn thiện trace và verifier;
- tạo nhật ký bài học;
- chạy lại toàn bộ bộ đánh giá;
- ghi danh sách giới hạn đã biết.

**Dấu hiệu hoàn thành:** người khác có thể dùng Laplace trong phạm vi hẹp, hiểu lúc nào cần duyệt và đọc được bằng chứng của kết quả.

# 12. Bộ bài tập học hiểu

Các bài này không yêu cầu xây hệ thống lớn. Mục tiêu là biến khái niệm thành trực giác.

## Bài 1 - Chatbot hay Agent?

Lấy năm tính năng dự kiến của Laplace. Với mỗi tính năng, đánh dấu:

- chỉ tạo nội dung;
- cần đọc dữ liệu thật;
- cần hành động;
- cần quan sát phản hồi;
- cần nhiều bước.

Nếu chỉ tạo nội dung, có thể chưa cần Agent loop.

## Bài 2 - Thiếu bộ não, mắt hay tay?

Chuẩn bị ba lỗi:

- mô hình chọn sai;
- thiếu dữ liệu liên quan;
- công cụ không làm được việc.

Phân loại nguyên nhân trước khi sửa. Bài này giúp tránh đổ mọi lỗi cho mô hình.

## Bài 3 - Bằng chứng hoàn thành

Viết 10 câu "đã xong" của Laplace. Với mỗi câu, ghi bằng chứng máy có thể kiểm tra. Nếu không tìm được bằng chứng, tiêu chí hoàn thành vẫn còn mơ hồ.

## Bài 4 - Nén mà không mất lý do

Cho Laplace một trace dài. Yêu cầu tóm tắt nhưng bắt buộc giữ:

- mục tiêu;
- quyết định;
- lý do;
- lỗi;
- nguồn;
- bước tiếp theo.

Kiểm tra xem bản tóm tắt có đủ để tiếp tục nhiệm vụ không.

## **Bài 5 - Thiết kế công cụ chống dùng sai**

Tạo hai phiên bản cùng một công cụ: một phiên bản mơ hồ và một phiên bản có tham số, quyền, ví dụ và lỗi rõ. Chạy cùng 10 yêu cầu rồi so sánh.

## **Bài 6 - Memory có quyền được quên**

Yêu cầu Laplace:

- liệt kê memory liên quan;
- giải thích vì sao từng mục được dùng;
- sửa một mục;
- xóa một mục;
- chứng minh mục đã xóa không còn xuất hiện.

## **Bài 7 - Tấn công bằng tài liệu**

Đặt một câu lệnh giả trong tài liệu tham khảo. Laplace phải coi nó là dữ liệu, không phải chỉ dẫn có quyền cao hơn. Không dùng dữ liệu thật hoặc hành động thật khi thử.

## **Bài 8 - Một Agent so với hai Agent**

Chọn một nhiệm vụ có phép xác minh thật. Chạy:

- một Agent làm toàn bộ;
- một Agent làm, một verifier kiểm tra bằng dữ liệu ngoài.

So sánh chất lượng và chi phí. Chỉ giữ multi-Agent nếu lợi ích đo được lớn hơn độ phức tạp.

## 13. Checklist trước khi gọi Laplace là "Agent"

---

### Phạm vi

- ☐ Có một nhóm người dùng rõ.
- ☐ Có loại nhiệm vụ rõ.
- ☐ Có tiêu chí hoàn thành kiểm tra được.
- ☐ Có danh sách việc Laplace không làm.

### Context

- ☐ Quy tắc ổn định tách khỏi lịch sử nhiệm vụ.
- ☐ Context chỉ chứa thông tin liên quan.
- ☐ Kết quả công cụ dài được lưu ngoài và có tóm tắt.
- ☐ Tóm tắt giữ quyết định, lý do và nguồn.

### Tools

- ☐ Tên và tham số dễ hiểu.
- ☐ Công cụ đọc và ghi được phân biệt.
- ☐ Kết quả và lỗi có cấu trúc.
- ☐ Quyền mặc định là tối thiểu.
- ☐ Hành động quan trọng có bước xác nhận.

### Memory và dữ liệu

- ☐ Có tiêu chí ghi memory.
- ☐ Memory có nguồn, ngày và cách xóa.
- ☐ Dữ liệu nhạy cảm không bị lưu thừa.
- ☐ Nội dung bên ngoài không tự sửa memory.

### Vòng lặp

- ☐ Có giới hạn bước, thời gian và chi phí.
- ☐ Có điều kiện dừng rõ.
- ☐ Có cơ chế tránh lặp cùng một lỗi.
- ☐ Có đường chuyển cho người dùng.

### Xác minh

- ☐ Mỗi nhiệm vụ có bằng chứng hoàn thành.
- ☐ Không tin lời tự khai "đã xong".
- ☐ Có bộ đánh giá lặp lại được.
- ☐ Mọi thay đổi quan trọng được chạy lại bộ đánh giá.

## An toàn

- ☐ Hành động được phân loại theo rủi ro.
- ☐ Có bảo vệ prompt injection ở cả context và thực thi.
- ☐ Bí mật không xuất hiện trong log.
- ☐ Thao tác khó hoàn tác bị chặn hoặc yêu cầu duyệt.

## Vận hành

- ☐ Trace đủ để điều tra lỗi.
- ☐ Có phiên bản cho prompt, công cụ và memory schema.
- ☐ Có cách quay lui.
- ☐ Có danh sách giới hạn đã biết.

Nếu nhiều ô chưa được đánh dấu, Laplace vẫn có thể là một thử nghiệm tốt. Chỉ cần gọi đúng tên: prototype, chưa phải hệ thống tự chủ đáng tin.

# 14. Từ điển ngắn

**AI Agent:** hệ thống dùng mô hình để quyết định và dùng công cụ để hoàn thành công việc qua nhiều bước.

**LLM:** mô hình ngôn ngữ lớn, phần "bộ não" tạo và lựa chọn hành động.

**Prompt:** chỉ dẫn gửi cho mô hình. Prompt chỉ là một phần của context.

**Context:** toàn bộ thông tin mô hình nhìn thấy tại một thời điểm.

**Context window:** giới hạn lượng thông tin có thể đưa vào một lần gọi mô hình.

**Memory:** dữ liệu được lưu để dùng qua nhiều phiên.

**RAG:** tìm tài liệu liên quan rồi đưa vào context trước khi trả lời.

**Tool:** khả năng bên ngoài mà Agent có thể gọi, như tìm kiếm, đọc tệp hoặc API.

**Tool calling:** mô hình chọn công cụ và tạo tham số có cấu trúc.

**MCP:** chuẩn kết nối Agent với công cụ và nguồn dữ liệu.

**ReAct:** vòng lặp suy nghĩ, hành động và quan sát.

**Trajectory/Trace:** dấu vết các bước Agent đã đi qua.

**Harness:** hệ thống bao quanh mô hình để cung cấp context, tools, ràng buộc, xác minh và phục hồi.

**Guardrail:** rào chắn giới hạn hành vi hoặc yêu cầu xác nhận.

**Verifier:** bộ kiểm tra kết quả dựa trên dữ liệu hoặc quy tắc độc lập.

**Evaluation/Eval:** bộ nhiệm vụ và chỉ số dùng để đo Agent.

**Observability:** khả năng quan sát trạng thái, hành động, lỗi và chi phí của hệ thống.

**Sandbox:** môi trường cách ly để chạy mã hoặc thao tác có rủi ro.

**Multi-Agent:** nhiều Agent phối hợp bằng vai trò và cơ chế giao tiếp rõ.

**Prompt injection:** nội dung không đáng tin cố làm Agent hiểu nhầm dữ liệu thành mệnh lệnh.

## 15. Nguồn và cách trích dẫn

Tài liệu này là bản tóm lược học tập độc lập, không phải bản dịch chính thức hoặc bản thay thế cuốn sách gốc. Các câu trích ngắn được đặt trong ngoặc kép; phần còn lại là diễn giải và áp dụng cho Laplace.

Nguồn được đối chiếu tại commit 6f9132df8637df3c2350379d790ae9fc89dce3c9 của repo gốc, ngày 28/07/2026.

### [1] Tổng quan và bản tiếng Việt

Li Bojie, "Hiểu sâu về AI Agent: Nguyên lý thiết kế và thực hành kỹ thuật", README tiếng Việt. [Mở nguồn](#)

### [2] Chương 1 - Nền tảng Agent và Harness Engineering

Công thức Agent, vòng lặp ReAct, ràng buộc, xác minh, sửa lỗi và ba nguyên tắc thiết kế. [Mở nguồn](#)

### [3] Chương 2 - Context Engineering

Cấu trúc context, system prompt, kỹ năng, prompt injection và nén context. [Mở nguồn](#)

### [4] Chương 3 - Memory và kho kiến thức

Bộ nhớ người dùng, RAG, truy xuất lai, chỉ mục có cấu trúc và quản trị kiến thức. [Mở nguồn](#)

### [5] Chương 4 - Công cụ

Thiết kế công cụ, MCP, công cụ nhận biết/thực thi/cộng tác và kiến trúc theo sự kiện. [Mở nguồn](#)

### [6] Chương 5 - Coding Agent và an toàn

Mô hình đe dọa, sandbox, kiểm soát mạng, hệ thống tệp và memory qua phiên. [Mở nguồn](#)

### [7] Chương 6 - Đánh giá Agent

Thiết kế nhiệm vụ, chỉ số, chi phí, ý nghĩa thống kê, observability và cải tiến dựa trên đánh giá. [Mở nguồn](#)

### [8] Chương 8 - Sự tiến hóa liên tục

Kết tinh kinh nghiệm thành tri thức, chỉ dẫn, chương trình và vòng phát hành có xác minh. [Mở nguồn](#)

### [9] Chương 10 - Cộng tác Multi-Agent

Chia sẻ context, cấu trúc cộng tác, bộ xác minh và các kiểu lỗi của nhiều Agent. [Mở nguồn](#)

### [10] Gợi ý học tập của dự án

Lộ trình từ nền tảng, context, tools, evaluation tới tự tiến hóa và multi-Agent. [Mở nguồn](#)

### [11] Giấy phép

Repo gốc phát hành theo Apache License 2.0; một số dự án con có thể có giấy phép riêng. [Mở giấy phép](#)

## Ghi chú về link ban đầu

Repo tothanhnghuyen/ai-agent-book mà bạn gửi là một bản fork của bojieli/ai-agent-book. Sổ tay dùng repo gốc làm nguồn để nhận bản cập nhật mới hơn.

## Lời kết cho Laplace

Đừng bắt đầu bằng câu hỏi "làm sao để Laplace tự chủ hơn?". Hãy bắt đầu bằng ba câu:

- 1 Laplace cần nhìn thấy gì để quyết định đúng?
- 2 Laplace được phép làm gì và bằng công cụ nào?
- 3 Bằng chứng nào cho thấy việc đã hoàn thành thật?

Trả lời tốt ba câu đó, rồi thêm ràng buộc, xác minh và phục hồi. Đó là con đường ngắn nhất từ một demo biết nói tới một Agent có thể tin dùng.